

## 3 · MULTICORE

## Two cores: SMP on the ESP32

ESP32 has a dual-core Xtensa LX6: **core 0** (PRO\_CPU) and **core 1** (APP\_CPU). With **SMP** (Symmetric Multiprocessing) one scheduler sees both cores and runs two tasks truly in parallel.

### LET THE SCHEDULER CHOOSE A CORE

```
xTaskCreate(  
    sensorTask,    // function  
    "sensor",      // name  
    4096,          // stack (bytes)  
    NULL,          // arg  
    5,             // priority  
    NULL);         // handle
```

### PIN A TASK TO A CORE

```
xTaskCreatePinnedToCore(  
    mlTask, "ml", 8192,  
    NULL, 1, NULL,  
    0);    // core 0  
xTaskCreatePinnedToCore(  
    controlTask, "ctrl", 4096,  
    NULL, 10, NULL,  
    1);    // core 1
```

Typical split: heavy ML task on one core, real-time control loop on the other. Note: the Wi-Fi/BT stack runs on core 0 by default. Parallel tasks sharing data is exactly where the next problem begins.

- **Affinity:** an unpinned task may run on either core (`tskNO_AFFINITY`) and can move between them. Pin a task to keep its data warm in one core's cache and isolate it from other work.
- **Priority is global.** Each core runs the highest-priority Ready task it is allowed to run. Higher number = higher priority.
- **Stack size** is in bytes in ESP-IDF. Too small means a stack-overflow crash; check with `uxTaskGetStackHighWaterMark()`.